GESTURE-BASED-DRONE-CONTROL

Making drone control intuitive and accessible

CAPSTONE DEMO 1

Presented by: Ayush Beekum, Shavir Vallabh, Jaitin Moodally, Diya Narotam and Chinmayi Santhosh

AGENDA

- The problem
- Our Solution
- Architecture Overview
- Domain Model
- Product Plan
- Live Demo
- CI/CD
- Testing

THE PROBLEM?

Traditional drone controllers create barriers. Our solution removes them.

Current state

Controllers require memorization, physical hardware familiarity, steep learning curve

Who suffers

Educators, beginners, accessibility-focused users, people in hands-free environments

Gap

No intuitive, controller-free alternative exists

OUR SOLUTION

Live camera feed → Hand detection (MediaPipe) → Gesture recognition → Drone commands

Real Time Gesture Interpretation

take-off, landing, directional movement

Safety-critical features:

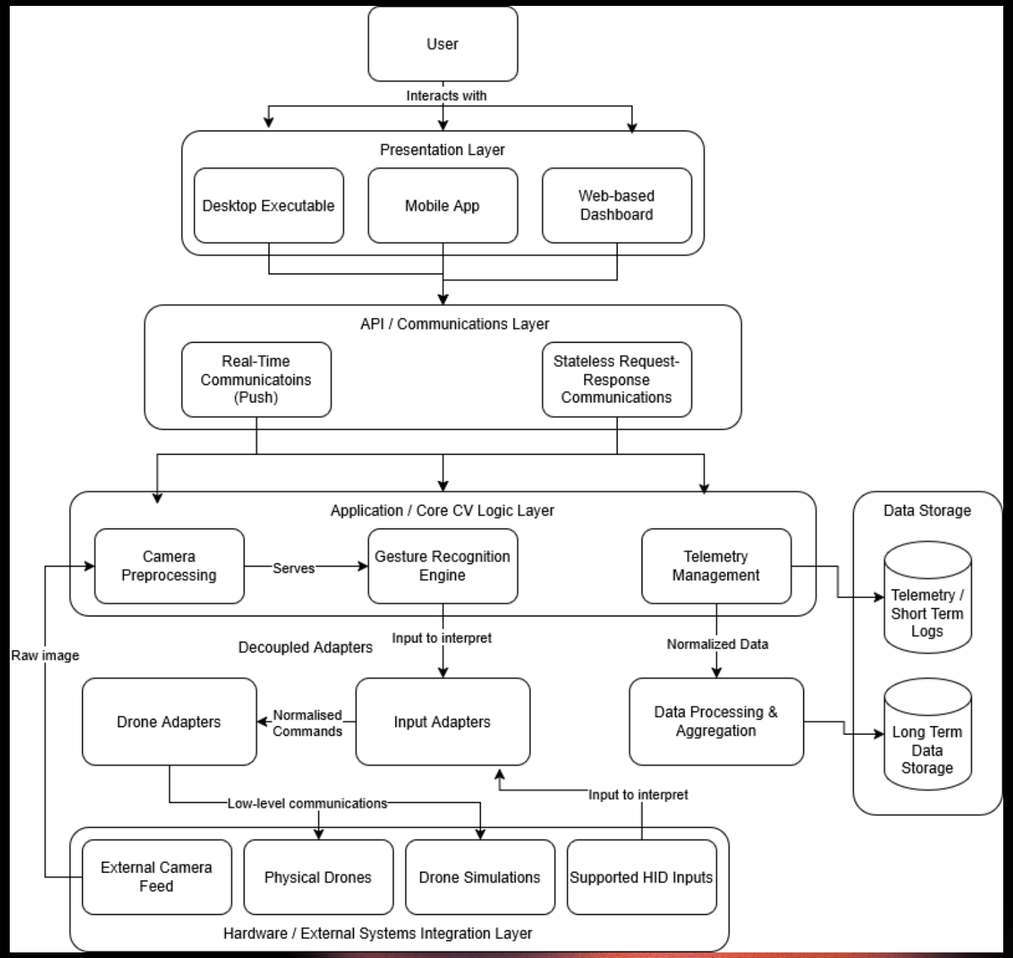
signal-loss hovering, emergency override

Platform-agnostic:

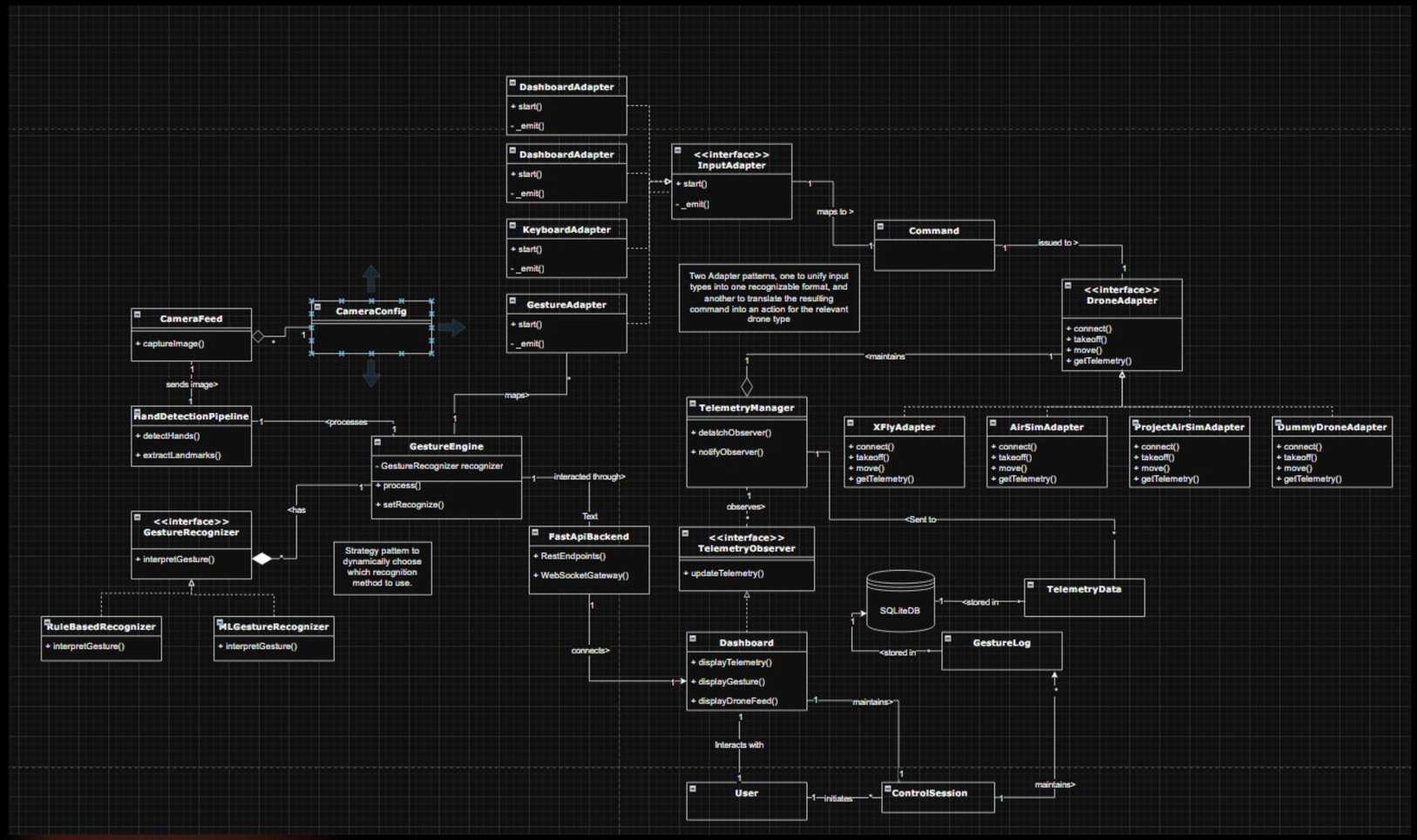
Desktop (Electron), Mobile (PWA + Capacitor)

ARCHITECTURE OVERVIEW

- **Layered Architecture** – Separates presentation, core application logic, and hardware control
- **Central CV Logic Layer** – Handles camera input interpretation (to drone) and telemetry data collection
- **Hexagonal Architecture Influence** – Isolates hardware layers and external frameworks
- **Key Outcomes** – Safe camera input pipelines, decoupled I/O, and improved extensibility



DOMAIN MODEL



PRODUCT PLAN

UC-1: Hand Tracking

- Core foundation (Demo 1 focus)
- Deliverable: Real-time hand landmarks
- Technology: MediaPipe Hands + OpenCV preprocessing
- Measurement: Confidence, latency and FPS
- Demo outcome: Live camera feed with async queue smooth tracking

PRODUCT PLAN

UC-2: Dashboard Telemetry

- Mock telemetry data and visualization (Demo 1 focus)
- Deliverables:
  - FastAPI backend with WebSocket communication
  - React dashboard displaying live drone state (altitude, battery, position)
  - Telemetry data flow: Simulator → Backend → Frontend
  - Command history log (timestamp-sorted, latest commands visible)
- Testing: unit testing done using Playwright
- Demo outcome: Mock telemetry data displayed on a dashboard interface

PRODUCT PLAN

UC-3: Drone Adapter

- Hardware abstraction layer to interface input methods and drone models
- Deliverables:
- Two-way adapter patterns to generalize communication
- Every single input method works with every single drone implementation
- Allows parity between flying the real drone and using one of the sims
- Implemented using a unified Command system, sitting in between abstract DroneAdapter and InputAdapter interfaces
- This decoupled architecture loosely follows the Hexagonal architecture pattern (Ports & Adapters)

CODEXMERCHANTS

LIVE DEMO

CI/CD

CI/CD pipeline (GitHub Actions)

Linting

Trigger : Pull request to any of the branches:

- lint backend
- lint services
- lint frontend

steps:
1)install dependencies
2)runs linter(prettier/ruff)

Testing

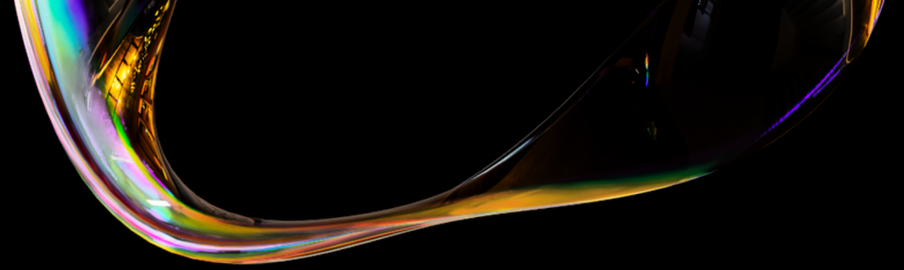
Trigger : Pull request to Dev and Main:

- test backend
- test services
- test frontend

steps:
1)install dependencies
2) runs tests(Pytest/Playwright)

Docs

Trigger : Pull request to Dev and Main
steps:
1)installs dependencies
2) Deploy to GitHub Pages



TESTING

Backend Testing (Pytest):

- Hand detection unit tests: Verify landmark detection through mocked camera usage
- Gesture recognition tests: Verify each gesture recognized with confidence
- API endpoint tests: REST (flight summary) and WebSocket (telemetry streaming)

Frontend Testing (Playwright)

Unit Tests:
It is separated into 3 levels:

- testing the atoms
- testing the molecules
- testing the organisms

Each level tests cover rendering, interactions,clickability, display and navigation.

THANK YOU

Questions?

codex.merchants@gmail.com